Project Proposal & Technical Blueprint

1. Group Information

Group Number: 7
Binny Bajaj, Michael Miranda, Vincent Hayes Bursey
Project Topic - Transportation and Urban Mobility Analytics

2. Business Problem

Taxi companies often measure profitability by using total trips or total revenue; however, these measures do not fully show if the vehicles have been operating efficiently. Some trips might cover the areas with limited demand for few passengers, but others may generate high fares but need more distance or time, so this can increase the driver's waiting time, vehicle repositioning and potential revenue loss.

This project will utilize the New York taxi trips data by taking a few years into account to identify the zones, routes and time periods that provide the passenger demand and strongest revenue efficiency. The analysis will not only use measures such as total revenue, fare amount, trips distance, duration, revenue per mile and revenue per minute, but also will compare the pick-up and drop-off activity by zone to identify areas where vehicle may face risk of idle time and repositioning.

The main question that this project will delve into:

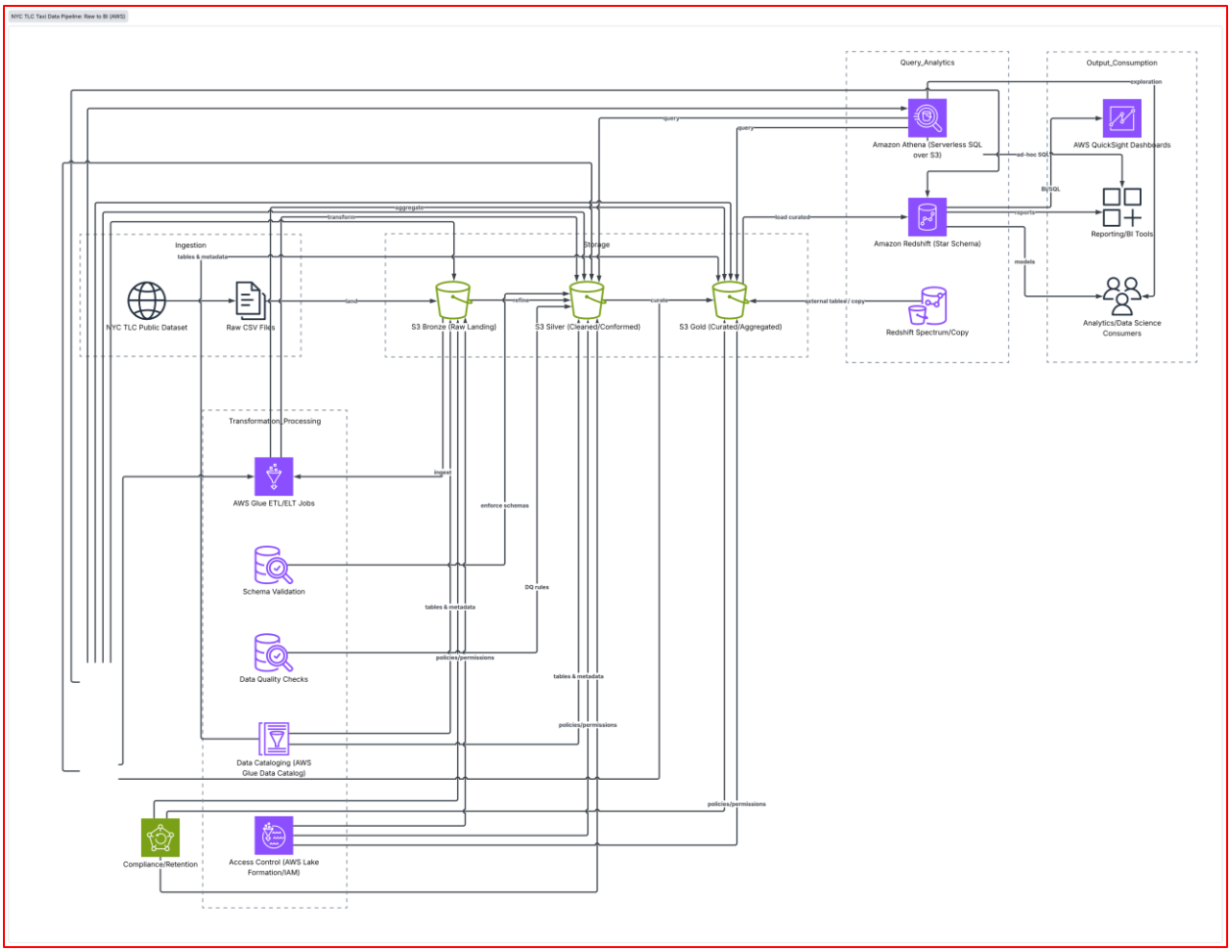
How can taxi companies improve revenue efficiency and vehicle positioning by analyzing the locations, time periods, and trip patterns that create high passenger demand and generate strong revenue for the business?

The intended stakeholders or users are taxi company operations managers, infrastructure managers, data engineering teams, revenue analysts, and drivers. These stakeholders can use the insights to make informed decisions regarding vehicle placement, identify the high-demand areas, and enhance the overall revenue efficiency.

This problem is relevant from a business analytics perspective because this combines all attributes such as financial, operational, geographic, and time-based data. This can reveal the seasonal trends, demand patterns, high-performing zones, and possible operational efficiencies by analyzing the few years of trip records which may not be visible from individual trips.

3. **Dataset Plan**
    a. What dataset will you use?
        i. NYC Yellow Taxi Trip Data – Jan, Feb and March of 2016
    b. Where does the data come from?
        i. This taxi trip data originates from the NYC Taxi and Limousine Commission, which publicly releases yellow taxi trip information in the boroughs of NYC.
        ii. The data used in the attached datasets were collected and provided to the NYC Taxi and Limousine Commission (TLC) by technology providers authorized under the Taxicab & Livery Passenger Enhancement Programs (TPEP/LPEP).
        iii. TLC relies on an external provider to provide the data and has verified that these external providers are authorized under the TPEP/LPEP (Taxicab & Livery Passenger Enhancement Program).
        iv. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
    c. What type of data is it: structured, semi-structured, streaming, transactional, text, etc.?
        i. This specific dataset contains transactional and structured data. Each row signifies a completed taxi trip with around 19 columns.

    d. What are the main fields, entities, or variables you expect to work with?
        i. Timestamps: tpep_pickup_datetime, tpep_dropoff_datetime: Engagement and disengagement of meter
        ii. Trip Describers: passenger_count, trip_distance (distance in miles), ratecodeid (standard rate, airport/JFK, negotiated fare, etc.)
        iii. Locational/Geographical: pickup_longitude / pickup_latitude, dropoff_longitude/dropoff_latitude
        iv. Payment Information: payment_type, fare_amount, tip_amount, tolls_amount, total_amount
4. **Proposed Cloud Architecture**
    a. Identify the main AWS services you expect to use.
    b. You may also include a brief GCP comparison if relevant.



- Ingestion Layer: Raw CSV landing zone in Amazon S3 standard storage (s3://nyc-taxi-lake/raw/).
- Processing & Catalog Layer: Serverless PySpark via AWS Glue for data validation, schema enforcement, and outputting dynamic Parquet partitions (s3://nyc-taxi-lake/curated/). AWS Glue Data Catalog for metadata auto-crawling.
- Analytical & Query Layer: Amazon Athena for serverless SQL analytics; Amazon Redshift Serverless for star schema data warehouse queries (fact_trips joined with dimensional tables)
- Consumption & Visualization Layer: Jupyter Notebook environment using Python (pandas, plotly, matplotlib) for RFM heatmaps and route density visualizations.
- GCP Benchmarking Overlay: Comparative mapping showing how Google Cloud Storage (GCS), Dataproc, and BigQuery map to each phase.

Pipeline Stage	Primary AWS Service	Comparative GCP Service	Core Response
Object Storage	Amazon S3	Google Cloud Storage	Host raw landing files & processed Parquet partitions
Metadata / Catalog	AWS Glue Data Catalog	GCP Dataplex / BigQuery Metadata	Store schema definitions & partition locations
ETL / Transformation	AWS Glue (PySpark)	GCP Dataproc Serverless	Execute data cleaning, type casting, and Parquet partitioning
Serverless Query	Amazon Athena	Google Cloud BigQuery	Run benchmark queries, measure scanned data volume & cost
Data Warehouse	Amazon Redshift Serverless	Google Cloud BigQuery Native	Model star schema for complex analytical joins
Visualization / Code	Jupyter / Python (Plotly)	Vertex AI Workbench / Jupyter	Build RFM segment plots and cost-performance charts

Risk A: Geographic & Operational Anomaly Skewing Distance / Fare Calculations

Risk B: Data Skewing & Small File Problem in Athena / S3 Partitioning

5. Analytics Output

The final output will be an interactive dashboard supported by SQL analysis and brief quality summary. This will display the key metrics such as Total trips, total revenue, average trip distance, average trip duration, revenue per mile, and revenue per minute, and along with that will show trends like high-performing zones, time periods, and routes.

Another section of the dashboard will show comparison of pickup and drop-off activities by zones to identify the higher possibility of driver waiting time or vehicle repositioning to make relevant decisions.

The Amazon Athena SQL queries will be performed to calculate these performance measures, and it will also include short data-quality summary focusing on covering missing values, invalid fares, zero or negative distance, etc.

6. Risks and Challenges

While initial risks like cost overruns and OOM errors are standard, a dataset of 12M trip records introduces specific data-quality and architectural edge cases that need explicit mitigation strategies.

Risk Category	Potential Impact	Mitigation Strategy
Cost Overruns	Unpartitioned full-table scans in Athena/BigQuery accumulating unexpected charges during exploratory SQL querying	Enforce the WHERE clause partition filters in Athena and set up AWS Budget alerts at $5 - $10 thresholds
Schema Instability	Unexpected null values or data type mismatches in raw Kaggle CSV files breaking ETL scripts	Implement explicit schema validation checks in PySpark before re-writing to Parquet
Memory Bottlenecks	Out-of-Memory errors during Spark group-by and aggregation transformations on 12M rows	Optimize Spark worker allocation (DPUs) and perform initial filtering before heavy aggregation joins
Access Control / IAM	Unauthorized bucket access or overly broad permissions	Apply the Principle of Least Privilege using targeted IAM roles for AWS Glue and Athena execution

7. Initial Cost and Cleanup Plan

Assumption: Dataset size ~2GB raw CSV (~350 MB after Snappy-Parquet compression)

Estimated Monthly AWS Usage & Cost Breakdown	
Amazon S3 Storage (2.5 GB total raw + processed)	$
AWS Glue ETL (PySpark job running ~10-15 minutes)	$
AWS Athena Queries (50 GB total data scanned)	$
Redshift Serverless	$
Total Estimated Cost	$

To ensure no unintended cloud charges accrue after evaluation, an explicit budget and decommissioning plan is essential:

Service Component	AWS Estimated Cost	GCP Estimated Cost	Cost Factors
Data Lake Storage	**~$0.06** (S3 Standard ~2.5 GB)	**~$0.05** (GCS Standard ~2.5 GB)	Standard tier storage per month
ETL / Processing	**~$0.44** (AWS Glue PySpark: 2 DPUs × 10 mins)	**~$0.35** (Dataproc Serverless Spark)	Compute runtime for cleaning & Parquet write
Serverless Querying	**~$0.25** (Athena: ~50 GB scanned in testing)	**~$0.31** (BigQuery: ~50 GB scanned @ $6.25/TB)	Scanned bytes during SQL benchmark testing
Cataloging / Metadata	**~$0.00** (Glue Catalog: First 1M objects free)	**~$0.00** (Dataplex / BigQuery Metadata free tier)	Schema registration
Total Estimated Run	**~$0.75 – $1.00**	**~$0.70 – $0.95**	**Well within AWS Free Tier / Learner Lab Limits**

Decommissioning & Infrastructure Clean-Up Protocol:

To avoid lingering charges, execute the following tear-down workflow:

1. Storage Deletion
    a. Empty S3 buckets via AWS or Console and then delete the bucket containers
2. Catalog Cleanup
    a. Delete registered tables and the custom database in the AWS Glue Data Catalog to prevent automated crawler re-runs
3. Compute Cleanup
    a. Ensure no active Glue Interactive Sessions or EMR / Redshift Serverless namespaces remain active
4. IAM Role Pruning
    a. Remove customer IAM roles developed for AWS Glue execution and Athena S3 full-access
5. Final Billing Validation
    a. Review AWS Cost Explorer post-cleanup to confirm active daily spend drops to $0